Forecast Inference for Time Series with TimeGrad Diffusion Model

Student : Zi-Wei Hong

Advisor : Prof. Nan-Jung Hsu

Institute of Statistics and Data Science, National Tsing Hua University

July 2, 2025

Outline

Motivation

- **Conventional time series models** usually require **stationary** assumption, e.g., VAR, transfer function model, ...
- **Deep learning models** such as RNN, LSTM **do not require** the series to be **stationary**.
- To **quantify the uncertainty** for deep learning models is **complicated**.
- Through **diffusion model**, one can generate samples and make inferences including **quantifying the uncertainty**.
- **Goal: Investigate the utility and limitation of diffusion models on time series forecasting**.

Denoising Diffusion Probabilistic Model (DDPM)

Denoising Diffusion Probabilistic Model (DDPM) consists of 2 components:

1. **Forward process**: Builds up the **connection from true data distribution to pure Gaussian noise** by gradually adding Gaussian noise to the observed data.
2. **Reverse process**: Tries to learn how to **recover the observed data** by removing the added noise.

Forward process is self-defined, reverse process is learned.

Notations for DDPM

Table: Notations for DDPM.

Notation	Explanation
$\boldsymbol{x}^0$	Observed data
$q(\cdot)$	True data distribution
α_k	Retained Variance
$\boldsymbol{x}^k$	Noisy data
$p_{\boldsymbol{\theta}(\cdot)}$	Approximated distribution

Forward Process

- Given $\boldsymbol{x}^0 \sim q(\boldsymbol{x}^0)$, the forward process is defined by a Markov chain:

$$q(\boldsymbol{x}^{1:K}|\boldsymbol{x}^0) = \prod_{k=1}^{K} q(\boldsymbol{x}^k|\boldsymbol{x}^{k-1}).$$

- Each $q(\boldsymbol{x}^k|\boldsymbol{x}^{k-1})$ is a **Gaussian distribution**:

$$q(\boldsymbol{x}^k|\boldsymbol{x}^{k-1}) \sim \mathcal{N}\left(\boldsymbol{x}^k; \sqrt{\alpha_k}\,\boldsymbol{x}^{k-1}, (1-\alpha_k)\mathbf{I}\right)$$

$$\Updownarrow$$

$$\boldsymbol{x}^k = \sqrt{\alpha_k}\,\boldsymbol{x}^{k-1} + \sqrt{1-\alpha_k}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

- $\alpha_k \in (0,1)$ controls **how much original signal remains** and **the variance of added noise**.

Forward Process (Cont.)

- In practice, one can directly consider

$$q(\boldsymbol{x}^k|\boldsymbol{x}^0) \sim \mathcal{N}\left(\boldsymbol{x}^k; \sqrt{\tilde{\alpha}_k}\,\boldsymbol{x}^0, (1-\tilde{\alpha}_k)\mathbf{I}\right), \quad \tilde{\alpha}_k = \prod_{i=1}^{k} \alpha_i$$

$$\Updownarrow$$

$$\boldsymbol{x}^k = \sqrt{\tilde{\alpha}_k}\,\boldsymbol{x}^0 + \sqrt{1-\tilde{\alpha}_k}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

  by the additive property of Gaussian distribution instead of adding noise step-by-step.

- To make $q(\boldsymbol{x}^K|\boldsymbol{x}^0) \dot{\sim} \mathcal{N}(\mathbf{0}, \mathbf{I})$, $\tilde{\alpha}_K$ needs to be small enough.

Reverse Process

- The **reverse process** is also defined by a Markov chain:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}^{0:K}) = p(\boldsymbol{x}^K) \prod_{k=1}^{K} p_{\boldsymbol{\theta}}(\boldsymbol{x}^{k-1}|\boldsymbol{x}^k), \quad p(\boldsymbol{x}^K) \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

- Each $p_{\boldsymbol{\theta}}(\boldsymbol{x}^{k-1}|\boldsymbol{x}^k)$ tries to **remove** $\boldsymbol{\epsilon}$ from $\boldsymbol{x}^k$ by a **Gaussian distribution** with $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\cdot)$ and $\boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\cdot)$:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}^{k-1}|\boldsymbol{x}^k) \sim \mathcal{N}\left(\boldsymbol{x}^{k-1}, \boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}^k, k), \boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\boldsymbol{x}^k, k)\right).$$

- $\boldsymbol{\theta}$ denotes the learnable parameters in $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\cdot)$ and $\boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\cdot)$.

- Once the reverse process learned, one can generate samples starting from $p(\boldsymbol{x}^K) \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and follow $p_{\boldsymbol{\theta}}(\boldsymbol{x}^{k-1}|\boldsymbol{x}^k)$ to $p_{\boldsymbol{\theta}}(\boldsymbol{x}^0)$.

Training Procedure of DDPM

- Objective: maximizing the log-likelihood by Evidence Lower Bound (ELBO):

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}^0) \geq \mathbb{E}_{q(\boldsymbol{x}^{1:K}|\boldsymbol{x}^0)}\left[\log \frac{P(\boldsymbol{x}^K)\prod_{k=1}^{K} p_{\boldsymbol{\theta}(\boldsymbol{x}^{k-1}|\boldsymbol{x}^k)}}{\prod_{k=1}^{K} q(\boldsymbol{x}^k|\boldsymbol{x}^{k-1})}\right] = L(\boldsymbol{\theta})$$

- Through the derivations, Ho et al. (2020) proposed

$$L_{\mathsf{simple}}(\boldsymbol{\theta}) \equiv \mathbb{E}_{k,\boldsymbol{x}^0,\boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\underbrace{\sqrt{\tilde{\alpha}_k}\,\boldsymbol{x}^0 + \sqrt{1-\tilde{\alpha}_k}\,\boldsymbol{\epsilon}}_{\boldsymbol{x}^k},\, k)\right\|^2\right]$$

- $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\cdot)$ aims to predict $\boldsymbol{\epsilon}$ from $\boldsymbol{x}^k$ without knowing $\boldsymbol{x}^0$.

Sampling Procedure of DDPM

- During the derivation, $p_{\boldsymbol{\theta}}(\boldsymbol{x}^{k-1}|\boldsymbol{x}^k)$ tries to approximate $q(\boldsymbol{x}^{k-1}|\boldsymbol{x}^k, \boldsymbol{x}^0)$ by

$$\mathcal{N}\left(\boldsymbol{x}^{k-1}; \frac{1}{\sqrt{\alpha}_k}\left(\boldsymbol{x}^k - \frac{1-\alpha_k}{\sqrt{1-\tilde{\alpha}_k}}\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}^k, k)\right), \frac{(1-\alpha_k)(1-\tilde{\alpha}_{k-1})}{(1-\tilde{\alpha}_k)}\mathbf{I}\right)$$

- Thus the sampling procedure starts from $\hat{\boldsymbol{x}}^K \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$\hat{\boldsymbol{x}}^{k-1} \leftarrow \frac{1}{\sqrt{\alpha}_k}\left(\hat{\boldsymbol{x}}^k - \frac{1-\alpha_k}{\sqrt{1-\tilde{\alpha}_k}}\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\hat{\boldsymbol{x}}^k, k)\right) + \sqrt{\frac{(1-\alpha_k)(1-\tilde{\alpha}_{k-1})}{(1-\tilde{\alpha}_k)}}\boldsymbol{z}\mathbb{I}\{k \neq 1\},$$

where $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $k = K, K-1, \ldots, 1$.

Problem Formulation of TimeGrad

TimeGrad (Rasul et al., 2021) considers a multivariate series $\{\boldsymbol{x}_t^0 \in \mathbb{R}^D : t = 1, 2, \ldots, T\}$ and covariates $\{\boldsymbol{c}_t : t = 1, 2, \ldots, T\}$.

- Task: forecast $(\boldsymbol{x}_{t_0+1}^0, \ldots, \boldsymbol{x}_T^0)$ given $(\boldsymbol{x}_1^0, \ldots, \boldsymbol{x}_{t_0}^0)$ and $(\boldsymbol{c}_1, \ldots, \boldsymbol{c}_T)$ via the conditional distribution:

$$q(\boldsymbol{x}_{t_0+1:T}^0 | \boldsymbol{x}_{1:t_0}^0, \boldsymbol{c}_{1:T}).$$

- Encode the history information up to time t by RNN mechanism with hidden feature $\boldsymbol{h}_t$:

$$\boldsymbol{h}_t \leftarrow \mathsf{RNN}_{\boldsymbol{\theta}} \left(\begin{bmatrix} \boldsymbol{x}_t^0 \\ \boldsymbol{c}_t \end{bmatrix}, \boldsymbol{h}_{t-1} \right), \quad \boldsymbol{h}_0 = \boldsymbol{0}.$$

- Approximate $q(\boldsymbol{x}_{t_0+1:T}^0 | \boldsymbol{x}_{1:t_0}^0, \boldsymbol{c}_{1:T})$ by

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t_0+1:T}^0 | \boldsymbol{x}_{1:t_0}^0, \boldsymbol{c}_{1:T}) = \prod_{t=t_0+1}^{T} p_{\boldsymbol{\theta}}(\boldsymbol{x}_t^0 | \boldsymbol{x}_{1:t-1}^0, \boldsymbol{c}_{1:T}) = \prod_{t=t_0+1}^{T} p_{\boldsymbol{\theta}}(\boldsymbol{x}_t^0 | \boldsymbol{h}_{t-1}).$$

Training Procedure of TimeGrad

- Training objective: minimizing the negative log-likelihood:

$$-\sum_{t=t_0+1}^{T} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}_t^0 | \boldsymbol{h}_{t-1}).$$

- Similar to DDPM, the loss function leads to

$$\mathbb{E}_{k,\boldsymbol{x}_t^0,\boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\underbrace{\sqrt{\tilde{\alpha}_k}\,\boldsymbol{x}_t^0 + \sqrt{1-\tilde{\alpha}_k}\,\boldsymbol{\epsilon}}_{\boldsymbol{x}_t^k},\ \boldsymbol{h}_{t-1},\ k)\right\|^2\right]$$

Sampling Procedure of TimeGrad

1. Run RNN over $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{t_0}\}$ and $\boldsymbol{c}_{1:T}$ to get $\boldsymbol{h}_{t_0}$.

2. Sample $\hat{\boldsymbol{x}}_{t_0+1}^K \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

3. For $k = K, K-1, \ldots, 1$:

$$\hat{\boldsymbol{x}}_{t_0+1}^{k-1} \leftarrow \frac{1}{\sqrt{\alpha_k}} \left(\hat{\boldsymbol{x}}_{t_0+1}^k - \frac{1-\alpha_k}{\sqrt{1-\tilde{\alpha}_k}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\hat{\boldsymbol{x}}_{t_0+1}^k, \boldsymbol{h}_{t_0}, k) \right)$$

$$+ \sqrt{\frac{(1-\alpha_k)(1-\tilde{\alpha}_{k-1})}{(1-\tilde{\alpha}_k)}} \boldsymbol{z} \mathbb{I}\{k \neq 1\}.$$

Sampling Procedure of TimeGrad (Cont.)

4. Run RNN over $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{t_0}, \hat{\boldsymbol{x}}_{t_0+1}\}$ and $\boldsymbol{c}_{1:T}$ to get $\boldsymbol{h}_{t_0+1}$.

5. Sample $\hat{\boldsymbol{x}}_{t_0+2}^K \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

6. For $k = K, K-1, \ldots, 1$:

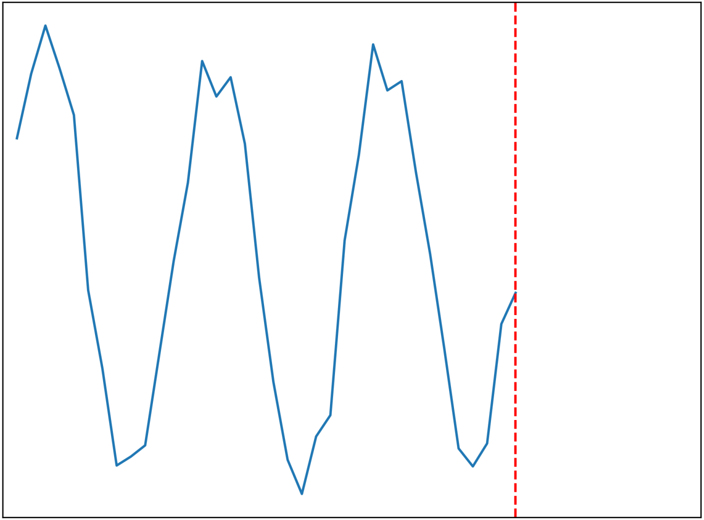
$$\hat{\boldsymbol{x}}_{t_0+2}^{k-1} \leftarrow \frac{1}{\sqrt{\alpha_k}} \left(\hat{\boldsymbol{x}}_{t_0+2}^k - \frac{1-\alpha_k}{\sqrt{1-\tilde{\alpha}_k}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\hat{\boldsymbol{x}}_{t_0+2}^k, \boldsymbol{h}_{t_0+1}, k) \right)$$

$$+ \sqrt{\frac{(1-\alpha_k)(1-\tilde{\alpha}_{k-1})}{(1-\tilde{\alpha}_k)}} \boldsymbol{z} \mathbb{I}\{k \neq 1\}.$$

7. Repeat 4 $\sim$ 6 until $\hat{\boldsymbol{x}}_T^0$ generated.

The sampling procedure can be repeated for S times.

Visualization of Sampling Procedure



Forecast Inferences for x_t

The sampling procedure can be repeated for S times, i.e., we will have $\{\hat{\boldsymbol{x}}^0_{t,s} : t = t_0 + 1, \ldots, T, s = 1, \ldots, S\}$.

To simplify the explanation, we first focus on the univariate case at time step t, i.e., $\{x_{t,s} \in \mathbb{R} : s = 1, \ldots, S\}$.

- **Forecast mean** for x_t:

$$\bar{x}_t = \frac{1}{S} \sum_{s=1}^{S} x_{t,s}$$

- For $\tau \in [0, 1]$, the $\mathbf{100\boldsymbol{\tau}\%}$**-th quantile** for x_t:

$$Q^*_t(\tau) = x_{t,([s\tau])}$$

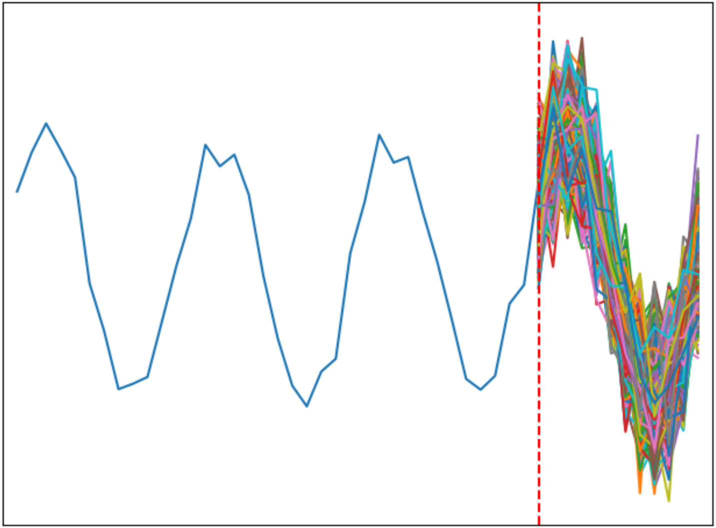
- **Forecast median** for x_t:

$$Q^*_t(0.5)$$

- $\mathbf{100\boldsymbol{\tau}\%}$ **forecast interval** for x_t:

$$\left(Q^*_t(\tfrac{1-\tau}{2}),\, Q^*_t(\tfrac{1+\tau}{2})\right)$$

Forecast Inferences from the Generated Samples



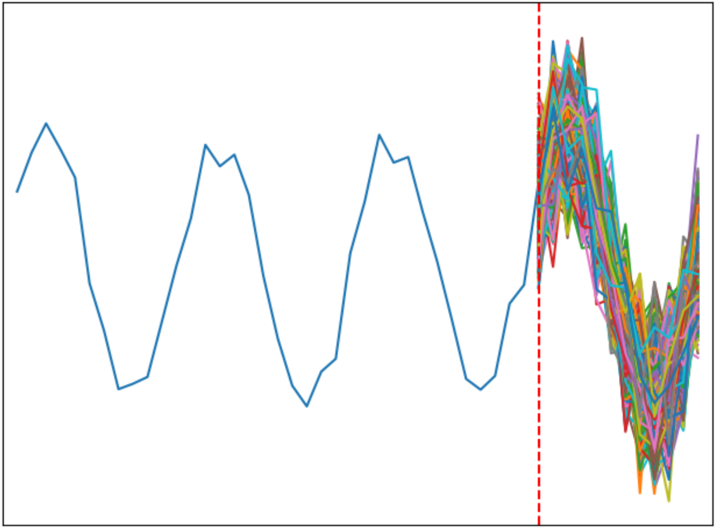
General Pareto Distribution and Extreme Quantiles

Under finite samples, the sample quantiles will be bounded by sample maximum and minimum.

To fix this issue, GPD is applied on:

1. $\{x_{t,s} : x_{t,s} > Q_t^*(0.975)\}$ and infer $\tilde{Q}_t(0.995)$ by GPD.
2. $\{x_{t,s} : x_{t,s} < Q_t^*(0.025)\}$ and infer $\tilde{Q}_t(0.005)$ by GPD.

GPD Quantile and Sample Quantile



Evaluation Metrics for Forecast Inferences

Given the generated samples $\{x_{t,s} : s = 1, \ldots, S\}$ and observation x_t:

1. CRPS: $\frac{1}{S}\sum_{i=1}^{S}|x_{t,i} - x_t| - \frac{1}{2S^2}\sum_{i=1}^{S}\sum_{j=1}^{S}|x_{t,i} - x_{t,j}|$
2. Square error: $(\bar{x}_t - x_t)^2$
3. Absolute error: $|Q_t^*(0.5) - x_t|$
4. $100\tau\%$ forecast interval width: $\left(Q_t^*(\frac{1+\tau}{2}) - Q_t^*(\frac{1-\tau}{2})\right)$
5. $100\tau\%$ forecast interval coverage: $\mathbb{I}\{x_t \in \left(Q_t^*(\frac{1-\tau}{2}), Q_t^*(\frac{1+\tau}{2})\right)\}$

Evaluation under TimeGrad Scenario

In TimeGrad, we will have $\{\boldsymbol{x}_{t,s} \in \mathbb{R}^D : t = t_0 + 1, \ldots, T, s = 1, \ldots, S\}$.

Each metric can be computed over all dimensions and $t = t_0 + 1, \ldots, T$.

To sum up the series, each metrics are computed as:

1. CRPS: sum over all dimensions and take average over $t = t_0 + 1, \ldots, T$
2. Square error: sum over all dimensions and take average over $t = t_0 + 1, \ldots, T$
3. Absolute error: sum over all dimensions and take average over $t = t_0 + 1, \ldots, T$
4. $100\tau\%$ forecast interval coverage: take average over all dimensions and $t = t_0 + 1, \ldots, T$

Implementation Details of TimeGrad

- $\alpha_1 = 0.9999$ to $\alpha_{100} = 0.9$ with linear schedule.
- Batch size $B = 64$.
- Learning rate $\lambda = 10^{-4}$.
- Number of generated samples $S = 1000$.
- Adam optimizer (Kingma and Ba, 2015) with $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\delta = 10^{-7}$.
- Train up to 50 epochs, early stopping after 5 epochs without improvement on validation.

Australia Electricity Dataset

- Target series: electricity demand in 5 states.
- $t_0 = 48 \times 7 = 336$ and $T = 48 \times (7 + 1) = 384$.

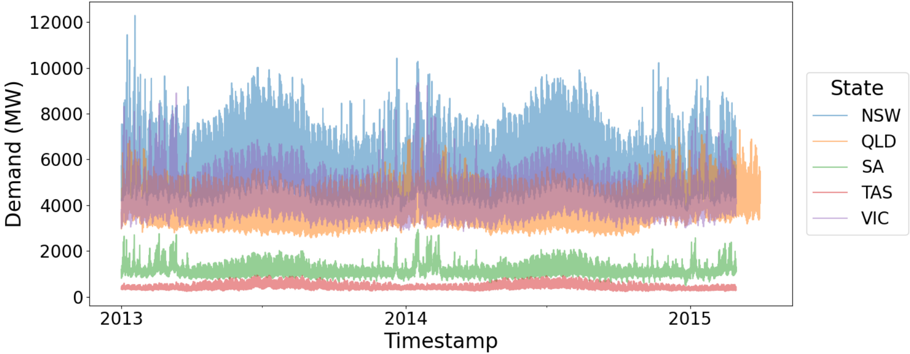
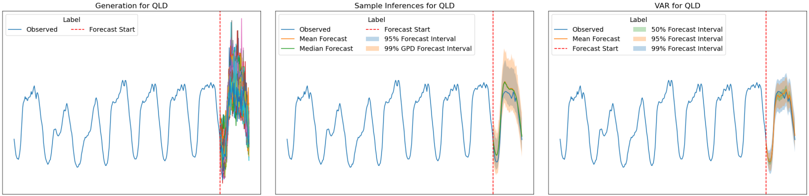


Figure: Electricity demands from 2013/01 to /2015/01.

TimeGrad and VAR on Australia Electricity Dataset



(a) Generated samples (b) Sample inferences (c) VAR results

Figure: Results of TimeGrad and VAR for Australia Electricity Dataset.

Evaluation for Australia Electricity Dataset

Table: Metrics on Australia Electricity Dataset.

Model	CRPS	Square Error	Absolute Error
VAR	458.84 ± 339.12	210695.19 ± 374241.49	458.84 ± 339.12
TimeGrad	964.51 ± 308.81	699474.36 ± 445859.12	1250.75 ± 348.32
Model	**50% Coverage**	**95% Coverage**	**99% Coverage**
VAR	0.58 ± 0.26	0.93 ± 0.13	0.97 ± 0.09
TimeGrad	0.28 ± 0.07	0.67 ± 0.09	0.79 ± 0.08
TimeGrad & GPD	–	–	0.80 ± 0.08

- VAR attains better forecast results.
- TimeGrad has lower coverage, is its forecast interval width narrower?

Forecast Interval Width for Australia Electricity Dataset

- Widths of VAR accumulate over time.
- Widths of TimeGrad hold constant over time.
- TimeGrad uses wider interval but gains lower coverage.

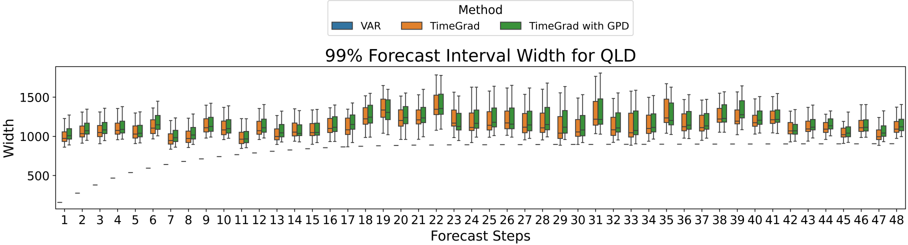
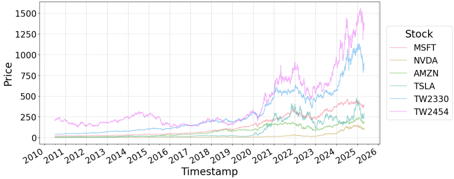


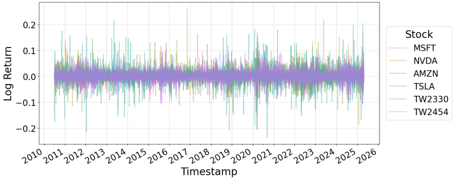
Figure: 99% forecast interval widths for Australia Electricity Dataset.

Stock Price Dataset

- Target series: stock price of 6 stocks.
- $t_0 = 5 \times 5 = 25$ and $T = 5 \times (5 + 1) = 30$.
- Model on prices and log returns.



(a) Prices



(b) Log returns

Figure: Stock prices and log returns in Stock Price Dataset.

Evaluation for Stock Price Dataset

Table: Metrics on Stock Price Dataset.

Model	Method	50% Coverage	95% Coverage	99% Coverage
VAR	Original	0.0373 ± 0.1123	0.0867 ± 0.2130	0.1020 ± 0.2384
	Reversed	0.5040 ± 0.3293	0.8667 ± 0.2295	0.9273 ± 0.1604
TimeGrad	Original	0.7413 ± 0.1831	0.9673 ± 0.0572	0.9913 ± 0.0265
	Reversed	0.8487 ± 0.0892	0.9987 ± 0.0093	1.0000 ± 0.0000
TimeGrad & GPD	Original	-	-	0.9913 ± 0.0273
	Reversed	-	-	1.0000 ± 0.0000

- Stock prices are nonstationary, which leads to the poor performance of VAR.
- Examine the forecast interval widths.

Forecast Interval Width for Stock Price Dataset

Table: Mean of Forecast Interval Widths on Stock Price Dataset.

Level	Model	Method	MSFT	NVDA	AMZN	TSLA	TW2330	TW2454
50%	VAR	Original	1.40	0.14	1.51	0.84	3.37	8.44
		Reversed	3.73	1.49	8.20	19.56	61.76	98.08
	TimeGrad	Original	96.14	9.41	25.78	30.97	76.90	186.04
		Reversed	73.68	5.69	16.40	23.29	62.31	164.07
95%	VAR	Original	4.07	0.39	4.40	2.43	9.78	24.51
		Reversed	10.85	4.34	23.86	56.99	180.11	286.52
	TimeGrad	Original	362.87	29.83	78.96	95.82	264.92	1037.43
		Reversed	350.26	19.98	57.98	84.60	301.04	1715.69
99%	VAR	Original	5.35	0.52	5.78	3.20	12.85	32.21
		Reversed	14.23	5.70	31.38	75.06	237.41	378.22
	TimeGrad	Original	768.81	50.77	121.02	138.42	492.37	3462.18
		Reversed	4350.53	38.55	100.25	154.51	727.07	1158677.59
	TimeGrad & GPD	Original	767.71	52.40	125.04	114.46	501.10	3312.44
		Reversed	5966.23	75.17	190.16	291.15	1558.55	7375186.44

- VAR has wider widths for the reversed price.
- TimeGrad is unstable on the reversed price.
- In general, the widths of TimeGrad is still wider than that of VAR.

Discussion

- This thesis investigates the utility and limitation of diffusion model on time series forecasting.
- **What leads to the poor performance of TimeGrad** ?
  1. RNN module cannot sum up the history information properly.
  2. The scarcity of data.
- 2 aspects that can be improved:
  1. Shorten the interval width.
  2. Speed up the sampling procedure.
- Future work: Is the usage of covariates identifiable ?


THANKS FOR LISTENING

References

Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR 2015*.

Rasul, K., Seward, C., Schuster, I., and Vollgraf, R. (2021). Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *International Conference on Machine Learning*, pages 8857–8868.

Model Architecture

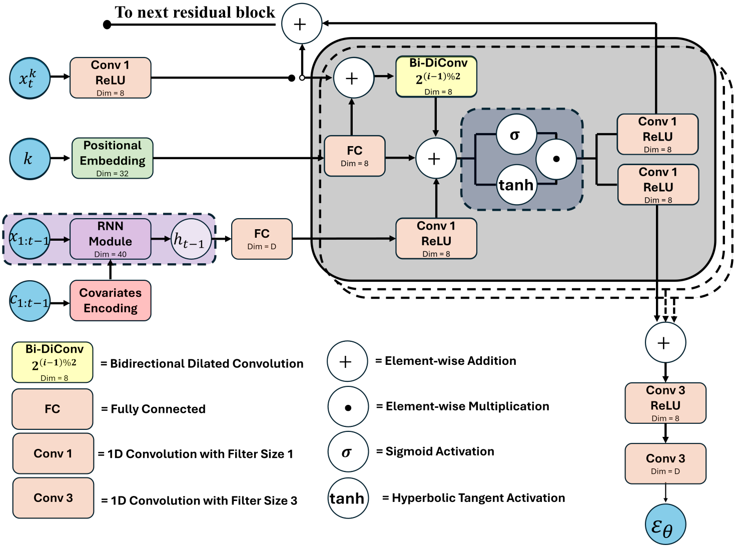


Figure: TimeGrad model architecture.

Execution time

Table: Execution time

	Australia Electricity		Stock Price	
Model	Training	Sampling	Training	Sampling
VAR	1:35:56	0:10	0:01	0:01
TimeGrad	34:51	1:23:20	1:17	5:50